

module-09-organizing-files

Chapter 9: Organizing Files | 23 programs in learning order

Source: *Automate the Boring Stuff with Python 1e*, Al Sweigart (CC-BY-NC-SA). Syllabus Ch9: *shutil copy/move, send2trash, os.walk, zipfile, rename-dates + backup-zip projects*

How to use this guide (beginner path)

1) Read Concepts once. 2) Study the Visuals - redraw each on paper. 3) Follow the Program map. 4) Work programs 01..N in order. [shell]: type every plain line into IDLE, compare with # OUT: comments (errors shown are intentional demos). [script]: save, predict output, run with F5. 5) Drill Interview prep. 6) Teach it: explain each visual aloud without looking.

Concepts in detail

This module moves paper around the office: `shutil.copy()/copytree()` duplicate, `shutil.move()` moves/renames, `os.unlink()/rmdir()/shutil.rmtree()` delete FOREVER (prefer `send2trash` for safety). `os.walk()` visits every subfolder - the engine behind bulk tools. `zipfile` opens/lists/extracts/creates zips. Two capstones combine everything: bulk date-format renaming with `regex`, and versioned whole-folder backup to zip.

Key ideas to carry forward

- `send2trash.send2trash(path)` deletes recoverably; `rmtree` does not.
- for folder, subs, files in `os.walk(root)`: is the standard traversal skeleton.
- `ZipFile('x.zip').extractall()` vs `.extract(name)`; write with `ZIP_DEFLATED` compression.

Classic beginner mistakes

- ! `rmtree` on the wrong path - no undo. Test prints before real deletes.
- ! Forgetting to close a `ZipFile` so the archive stays locked/incomplete.

Study order hint: 01-09 *copy/move/safe-delete*, 10-15 *walk + zip*, 16-23 the two capstone automations.

Interview prep - questions you must answer cold

Say each answer aloud in under 60 seconds, then prove it with one program from this module.

Q1. shutil vs os for file ops?

A: os handles paths/dirs basics; shutil copies whole trees (copytree), moves, and archives. Use shutil for bulk work.

Q2. Safe deletion practice?

A: Never rmtree on untested paths - no undo. Prototype with prints, then send2trash for recoverable deletes.

Q3. os.walk skeleton?

A: for folder, subs, files in os.walk(root): - visit everything; filter by extension inside.

Q4. zipfile read vs write?

A: ZipFile(z) + namelist()/extractall()/extract(name) to read; ZipFile(z,'w',ZIP_DEFLATED) + write(path) to create.

Q5. How would you rename 1000 dated files?

A: Regex the American date in each name, rebuild European name, os.rename - exactly the capstone project.

Q6. Versioned backup strategy?

A: Walk the tree, zip everything into backup_N.zip skipping prior backups - the second capstone.

Teach it to others

The one analogy that sticks

os.walk is a librarian walking every aisle (folder), noting shelves (subs) and books (files). shutil is the moving crew; send2trash is the recycling bin while rmtree is the shredder.

What your students will get wrong

- x move() only moves (it also renames).
- x extract() is always safe (zip-slip paths can escape - inspect names).
- x Deleted via Python goes to Recycle Bin (only send2trash does).

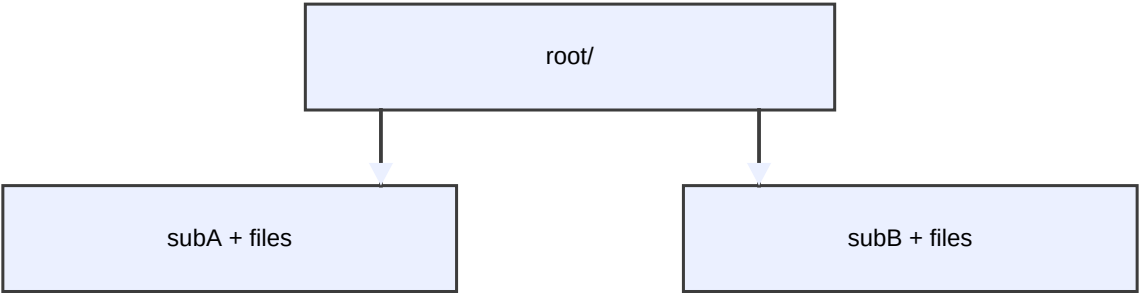
5-minute whiteboard plan

1. Draw the walk tree for a sample project folder.
2. Dry-run a rename with prints before real renames.
3. Students zip a test folder and list contents.
4. Exit ticket: write the walk skeleton from memory.

Visualize it - draw these from memory

Redraw each figure on paper with labels before you touch code. If you cannot draw it, you cannot teach it.

Visual 1 - os.walk visits every aisle



for folder, subs, files in walk(root): filter by extension inside, act on matches.

Visual 2 - zip in, zip out



Write with ZIP_DEFLATED; close the ZipFile or it stays locked. send2trash before rmtree, always.

More examples - same idea, new clothes

Three bonus drills per module. Cover the output, predict, then run.

X1. Tidy downloads by extension

```
import os, shutil
os.makedirs('pics', exist_ok=True)
for f in os.listdir('.'):
    if f.lower().endswith(('.png', '.jpg')):
        shutil.move(f, os.path.join('pics', f))
print('tidied')
# EXPECTED: Moves images into pics/
```

X2. Find large PDFs

```
import os
for folder, _, files in os.walk('.'):
    for f in files:
        if f.endswith('.pdf'):
            p = os.path.join(folder, f)
            if os.path.getsize(p) > 10**6:
                print(p)
# EXPECTED: Prints PDFs over 1MB
```

X3. Zip today's work

```
import zipfile, os
with zipfile.ZipFile('work.zip', 'w', zipfile.ZIP_DEFLATED) as z:
    for f in os.listdir('.'):
        if f.endswith('.py'):
            z.write(f)
print('zipped')
# EXPECTED: work.zip with all .py files
```

Program map - all 23 at a glance

```
01 01_copying_files_folders.py
02 02_copying_files_folders_shutil.py
03 03_moving_renaming_files.py
04 04_moving_renaming_files_shutil.py
05 05_moving_renaming_files_shutil_2.py
06 06_moving_renaming_files_shutil_3.py
07 07_permanently_deleting_files.py
08 08_permanently_deleting_files_os.py
09 09_safe_deletes_send2trash.py
10 10_walking_directory_tree.py
11 11_walking_directory_tree_the.py
12 12_reading_zip_files.py
13 13_extracting_from_zip.py
14 14_extracting_from_zip_examplezip.py
15 15_creating_adding_zip.py
16 16_step_1_create.py
17 17_step_2_identify.py
18 18_step_2_identify_datepattern.py
19 19_step_3_form.py
20 20_step_1_figure.py
21 21_step_2_create.py
22 22_step_3_walk.py
23 23_step_3_walk_creating.py
```

Topic: Copying Files and Folders

01/23 01_copying_files_folders.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import shutil, os
os.chdir('C:\\')
shutil.copy('C:\\spam.txt', 'C:\\delicious')
# OUT:      'C:\\delicious\\spam.txt'
shutil.copy('eggs.txt', 'C:\\delicious\\eggs2.txt')
# OUT:      'C:\\delicious\\eggs2.txt'
```

02/23 02_copying_files_folders_shutil.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
import shutil, os  
os.chdir('C:\\')  
shutil.copytree('C:\\bacon', 'C:\\bacon_backup')  
# OUT: 'C:\\bacon_backup'
```

Topic: Moving and Renaming Files and Folders

03/23 03_moving_renaming_files.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import shutil
shutil.move('C:\\bacon.txt', 'C:\\eggs')
# OUT: 'C:\\eggs\\bacon.txt'
```

04/23 04_moving_renaming_files_shutil.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
shutil.move('C:\\bacon.txt', 'C:\\eggs\\new_bacon.txt')
```

```
# OUT: 'C:\\eggs\\new_bacon.txt'
```

05/23 05_moving_renaming_files_shutil_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
shutil.move('C:\\bacon.txt', 'C:\\eggs')
```

```
# OUT: 'C:\\eggs'
```

06/23 06_moving_renaming_files_shutil_3.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
# Lines starting with >>> are input; other lines are expected output (commented).

shutil.move('spam.txt', 'c:\\does_not_exist\\eggs\\ham')
# OUT: Traceback (most recent call last):
# OUT:   File "C:\Python34\lib\shutil.py", line 521, in move
# OUT:     os.rename(src, real_dst)
# OUT: FileNotFoundError: [WinError 3] The system cannot find the path specified:
# OUT: 'spam.txt' -> 'c:\\does_not_exist\\eggs\\ham'

# OUT: During handling of the above exception, another exception occurred:

# OUT: Traceback (most recent call last):
# OUT:   File "<pyshell#29>", line 1, in <module>
# OUT:     shutil.move('spam.txt', 'c:\\does_not_exist\\eggs\\ham')
# OUT:   File "C:\Python34\lib\shutil.py", line 533, in move
# OUT:     copy2(src, real_dst)
# OUT:   File "C:\Python34\lib\shutil.py", line 244, in copy2
# OUT:     copyfile(src, dst, follow_symlinks=follow_symlinks)
# OUT:   File "C:\Python34\lib\shutil.py", line 108, in copyfile
# OUT:     with open(dst, 'wb') as fdst:
# OUT: FileNotFoundError: [Errno 2] No such file or directory: 'c:\\does_not_exist\\
# OUT: eggs\\ham'
```

Topic: Permanently Deleting Files and Folders

07/23 07_permanently_deleting_files.py [script]

Script: save and run; predict the output first, then compare.

```
import os
for filename in os.listdir():
    if filename.endswith('.rxt'):
        os.unlink(filename)
```

08/23 08_permanently_deleting_files_os.py [script]

Script: save and run; predict the output first, then compare.

```
import os
for filename in os.listdir():
    if filename.endswith('.rxt'):
        #os.unlink(filename)
        print(filename)
```

Topic: Safe Deletes with the send2trash Module

09/23 09_safe_deletes_send2trash.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import send2trash
baconFile = open('bacon.txt', 'a') # creates the file
baconFile.write('Bacon is not a vegetable.')
# OUT: 25
baconFile.close()
send2trash.send2trash('bacon.txt')
```

Topic: Walking a Directory Tree

10/23 10_walking_directory_tree.py [script]

Script: save and run; predict the output first, then compare.

```
import os

for folderName, subfolders, filenames in os.walk('C:\\delicious'):
    print('The current folder is ' + folderName)

    for subfolder in subfolders:
        print('SUBFOLDER OF ' + folderName + ': ' + subfolder)
    for filename in filenames:
        print('FILE INSIDE ' + folderName + ': ' + filename)

print('')
```

11/23 11_walking_directory_tree_the.py [script]

Script: save and run; predict the output first, then compare.

```
The current folder is C:\delicious
SUBFOLDER OF C:\delicious: cats
SUBFOLDER OF C:\delicious: walnut
FILE INSIDE C:\delicious: spam.txt
```

```
The current folder is C:\delicious\cats
FILE INSIDE C:\delicious\cats: catnames.txt
FILE INSIDE C:\delicious\cats: zophie.jpg
```

```
The current folder is C:\delicious\walnut
SUBFOLDER OF C:\delicious\walnut: waffles
```

```
The current folder is C:\delicious\walnut\waffles
FILE INSIDE C:\delicious\walnut\waffles: butter.txt.
```

Topic: Reading ZIP Files

12/23 12_reading_zip_files.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import zipfile, os
os.chdir('C:\\\\')    # move to the folder with example.zip
exampleZip = zipfile.ZipFile('example.zip')
exampleZip.namelist()
# OUT:      ['spam.txt', 'cats/', 'cats/catnames.txt', 'cats/zophie.jpg']
spamInfo = exampleZip.getinfo('spam.txt')
spamInfo.file_size
# OUT:      13908
spamInfo.compress_size
# OUT:      3828
'Compressed file is %sx smaller!' % (round(spamInfo.file_size / spamInfo
# OUT:      .compress_size, 2))
# OUT:      'Compressed file is 3.63x smaller!'
exampleZip.close()
```

Topic: Extracting from ZIP Files

13/23 13_extracting_from_zip.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import zipfile, os
os.chdir('C:\\')    # move to the folder with example.zip
exampleZip = zipfile.ZipFile('example.zip')
exampleZip.extractall()
exampleZip.close()
```

14/23 14_extracting_from_zip_examplezip.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
exampleZip.extract('spam.txt')  
# OUT: 'C:\\spam.txt'  
exampleZip.extract('spam.txt', 'C:\\some\\new\\folders')  
# OUT: 'C:\\some\\new\\folders\\spam.txt'  
exampleZip.close()
```

Topic: Creating and Adding to ZIP Files

15/23 15_creating_adding_zip.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import zipfile
newZip = zipfile.ZipFile('new.zip', 'w')
newZip.write('spam.txt', compress_type=zipfile.ZIP_DEFLATED)
newZip.close()
```

Topic: Step 1: Create a Regex for American-Style Dates

16/23 16_step_1_create.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# renameDates.py - Renames filenames with American MM-DD-YYYY date format
# to European DD-MM-YYYY.

import shutil, os, re

# Create a regex that matches files with the American date format.
datePattern = re.compile(r"^(.*?) # all text before the date
    ((0|1)?\d)-                # one or two digits for the month
    ((0|1|2|3)?\d)-            # one or two digits for the day
    ((19|20)\d\d)              # four digits for the year
    (.*?)$                     # all text after the date
    "", re.VERBOSE)

# TODO: Loop over the files in the working directory.

# TODO: Skip files without a date.

# TODO: Get the different parts of the filename.

# TODO: Form the European-style filename.

# TODO: Get the full, absolute file paths.

# TODO: Rename the files.
```

Topic: Step 2: Identify the Date Parts from the Filenames

17/23 17_step_2_identify.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# renameDates.py - Renames filenames with American MM-DD-YYYY date format
# to European DD-MM-YYYY.

--snip--

# Loop over the files in the working directory.
for amerFilename in os.listdir('.'):
    mo = datePattern.search(amerFilename)

    # Skip files without a date.
    if mo == None:
        continue

    # Get the different parts of the filename.
    beforePart = mo.group(1)
    monthPart  = mo.group(2)
    dayPart    = mo.group(4)
    yearPart   = mo.group(6)
    afterPart  = mo.group(8)

--snip--
```

18/23 18_step_2_identify_datepattern.py [script]

Script: save and run; predict the output first, then compare.

```
datePattern = re.compile(r"""^(1) # all text before the date
    (2 (3) )-           # one or two digits for the month
    (4 (5) )-           # one or two digits for the day
    (6 (7) )            # four digits for the year
    (8)$                # all text after the date
    """, re.VERBOSE)
```

Topic: Step 3: Form the New Filename and Rename the Files

19/23 19_step_3_form.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# renameDates.py - Renames filenames with American MM-DD-YYYY date format
# to European DD-MM-YYYY.

--snip--

    # Form the European-style filename.
    euroFilename = beforePart + dayPart + '-' + monthPart + '-' + yearPart +
                    afterPart

    # Get the full, absolute file paths.
    absWorkingDir = os.path.abspath('.')
    amerFilename = os.path.join(absWorkingDir, amerFilename)
    euroFilename = os.path.join(absWorkingDir, euroFilename)

    # Rename the files.
    print('Renaming "%s" to "%s"...' % (amerFilename, euroFilename))
    #shutil.move(amerFilename, euroFilename) # uncomment after testing
```

Topic: Step 1: Figure Out the ZIP File's Name

20/23 20_step_1_figure.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# backupToZip.py - Copies an entire folder and its contents into
# a ZIP file whose filename increments.

import zipfile, os

def backupToZip(folder):
    # Backup the entire contents of "folder" into a ZIP file.

    folder = os.path.abspath(folder) # make sure folder is absolute

    # Figure out the filename this code should use based on
    # what files already exist.
    number = 1
    while True:
        zipFilename = os.path.basename(folder) + '_' + str(number) + '.zip'
        if not os.path.exists(zipFilename):
            break
        number = number + 1

    # TODO: Create the ZIP file.

    # TODO: Walk the entire folder tree and compress the files in each folder.
    print('Done.')

backupToZip('C:\\delicious')
```

Topic: Step 2: Create the New ZIP File

21/23 21_step_2_create.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# backupToZip.py - Copies an entire folder and its contents into
# a ZIP file whose filename increments.

--snip--
    while True:
        zipFilename = os.path.basename(folder) + '_' + str(number) + '.zip'
        if not os.path.exists(zipFilename):
            break
        number = number + 1

    # Create the ZIP file.
    print('Creating %s...' % (zipFilename))
    backupZip = zipfile.ZipFile(zipFilename, 'w')

    # TODO: Walk the entire folder tree and compress the files in each folder.
    print('Done.')

backupToZip('C:\\delicious')
```

Topic: Step 3: Walk the Directory Tree and Add to the ZIP File

22/23 22_step_3_walk.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# backupToZip.py - Copies an entire folder and its contents into
# a ZIP file whose filename increments.

--snip--

# Walk the entire folder tree and compress the files in each folder.
for foldername, subfolders, filenames in os.walk(folder):
    print('Adding files in %s...' % (foldername))
    # Add the current folder to the ZIP file.
    backupZip.write(foldername)
    # Add all the files in this folder to the ZIP file.
    for filename in filenames:
        newBase = os.path.basename(folder) + '_'
        if filename.startswith(newBase) and filename.endswith('.zip'):
            continue # don't backup the backup ZIP files
        backupZip.write(os.path.join(foldername, filename))
backupZip.close()
print('Done.')
```



```
backupToZip('C:\\delicious')
```

23/23 23_step_3_walk_creating.py [script]

Script: save and run; predict the output first, then compare.

```
Creating delicious_1.zip...
Adding files in C:\delicious...
Adding files in C:\delicious\cats...
Adding files in C:\delicious\waffles...
Adding files in C:\delicious\walnut...
Adding files in C:\delicious\walnut\waffles...
Done.
```